

SPÉCIFICATIONS TECHNIQUES : MOTEUR DE SYNTHÈSE & FACTORY TA-LIB

RÉFÉRENCE : SPECS-ALGO-FACTORY-V1 PARENT : SAT_QHP_Vector_V1 DATE : 2026-01-12

STATUT : FROZEN

PARTIE A : SPÉCIFICATIONS ALGORITHMIQUES DU MOTEUR DE SYNTHÈSE (SAM)

Cette section détaille la logique de transformation des séries temporelles atomiques (5min) vers des fréquences agrégées (f_{target}) via compilation JIT (Numba), garantissant une intégrité causale absolue.

1. LOGIQUE DE RESAMPLING VECTORISÉ (NUMBA JIT)

Le moteur refuse l'utilisation de `pandas.resample()` en boucle critique pour des raisons de latence (overhead Python) et de gestion mémoire. L'algorithme opère directement sur les buffers NumPy contigus.

1.1. Définition Mathématique de l'Agrégation

Soit S_{src} la série source de fréquence f_{base} (5min) et S_{tgt} la série cible de fréquence f_{tgt} ($k \times f_{base}$). Pour chaque intervalle cible j , l'agrégation se définit par :

$$\begin{aligned} Open_j &= Open_{i_{start}} \\ High_j &= \max(High_{i_{start}}, \dots, High_{i_{end}}) \\ Low_j &= \min(Low_{i_{start}}, \dots, Low_{i_{end}}) \\ Close_j &= Close_{i_{end}} \\ Volume_j &= \sum_{k=i_{start}}^{i_{end}} Volume_k \end{aligned}$$

Où i_{end} est strictement défini par la clôture de la période, excluant tout tick appartenant à $j + 1$.

1.2. Pseudo-Code Numba (No-Python Mode)

```
@numba.njit(nogil=True, parallel=False)
def fast_resample_nb(timestamps, open, high, low, close, volume, target_period_ms)
    """
    Entrée: Arrays numpy (float64/int64) alignés.
    Sortie: Dictionnaire structuré OHLCV agrégé.
    """
    n_source = len(timestamps)
    # Estimation de la taille cible (Allocation prédictive)
    n_target_est = int((timestamps[-1] - timestamps[0]) / target_period_ms) + 1

    # Allocation Mémoire (Sortie)
```

```

out_ts = np.empty(n_target_est, dtype=np.int64)
out_open = np.empty(n_target_est, dtype=np.float64)
out_high = np.empty(n_target_est, dtype=np.float64)
out_low = np.empty(n_target_est, dtype=np.float64)
out_close = np.empty(n_target_est, dtype=np.float64)
out_vol = np.empty(n_target_est, dtype=np.float64)

curr_idx = 0
current_bucket_end = timestamps[0] - (timestamps[0] % target_period_ms) + tar

# Initialisation accumulateurs
acc_h = -1.0
acc_l = 1e18
acc_v = 0.0
open_captured = False

for i in range(n_source):
    ts = timestamps[i]

    # Détection changement de bin
    if ts >= current_bucket_end:
        # Clôture bougie précédente
        out_ts[curr_idx] = current_bucket_end
        out_high[curr_idx] = acc_h
        out_low[curr_idx] = acc_l
        out_close[curr_idx] = close[i-1] # Close du dernier tick valide
        out_vol[curr_idx] = acc_v

        # Reset accumulateurs
        curr_idx += 1
        current_bucket_end += target_period_ms
        while ts >= current_bucket_end: # Gestion des Gaps temporels (Bougies
            # Injection logique Fill (voir 2.1)
            current_bucket_end += target_period_ms

        open_captured = False
        acc_h = -1.0
        acc_l = 1e18
        acc_v = 0.0

    # Agrégation Intra-Barre
    if not open_captured:
        out_open[curr_idx] = open[i]
        open_captured = True

    acc_h = max(acc_h, high[i])
    acc_l = min(acc_l, low[i])
    acc_v += volume[i]

return (out_ts[:curr_idx], out_open[:curr_idx], ...)

```

2. GESTION DES VALEURS MANQUANTES (NAN) ET BIAIS

2.1. Protocole de Traitement des Gaps (NaN-Values)

Les gaps proviennent d'une absence de transactions sur f_{base} durant tout un intervalle f_{tgt} .

- **Règle de persistance prix :** `Forward Fill` (*ffill*).
 - $O_j = C_j = H_j = L_j = C_{j-1}$
- **Règle d'activité volume :** `Zero Fill` .
 - $V_j = 0$

2.2. Élimination du Biais d'Anticipation (Look-ahead Bias)

Le biais survient lorsque l'index temporel de la bougie agrégée correspond à son début (Open Time) mais que la donnée contient le Close.

- **Indexation stricte :** Le timestamp de l'index référentiel pour une bougie agrégée doit toujours être le **Close Time** (ou Open Time + Durée).
- **Alignement Vectorbt :** Lors de l'émission de signaux, utiliser `vbt.portfolio.from_signals(..., freq=target_freq)` . VBT gère intrinsèquement l'exécution au tick suivant l'index du signal.
- **Contrainte de Superposition :** Une donnée 1H calculée à 10:00 (clôture) ne doit être accessible aux algos 5min qu'à partir de l'index 10:00 (soit la bougie 10:00-10:05).

PARTIE B : DOCUMENTATION TECHNIQUE DE LA FACTORY TA-LIB (DTF)

Cette section définit l'architecture du pont dynamique entre les spécifications utilisateur (JSON/UI) et le moteur C optimisé de TA-Lib, encapsulé par Vectorbt Pro.

1. MÉCANISMES D'INTROSPECTION DYNAMIQUE

L'objectif est d'abstraire la bibliothèque TA-Lib pour ne pas coder en dur ("hardcode") les milliers de combinaisons possibles.

1.1. Extraction des Métadonnées

Utilisation des signatures de l'API Python Wrapper pour construire le catalogue.

- **Attributs Introspectés :**
 - `func_groups` : Mapping (ex: 'RSI' -> 'Momentum Indicators').
 - `func_flags` : Bitmask (Input flags, Output flags).
 - `func.__doc__` : Parsing Regex pour extraire les noms des paramètres (`optInTimePeriod` , etc.).

1.2. Mapping de Typage (Schema Translation)

Type TA-Lib (C)	Type UI (JSON Schema)	Type Python/Numpy	Contrainte Valeur
TA_Real	float (Slider)	<code>np.float64</code>	Range continu
TA_Integer	int (Stepper)	<code>np.int32</code>	Range discret

TA_Input_Real	DataSelector	np.ndarray (1D)	OHLCV Array
TA_Output_Real	PlotLine	np.ndarray (1D)	Résultat

2. ARCHITECTURE DE L'INDICATOR FACTORY (VECTORBT)

2.1. Classe `UniversalIndicator`

Création d'une classe usine générique héritant de `vbt.IndicatorFactory`.

```
Indicator = vbt.IndicatorFactory(
    class_name='DynamicTALib',
    short_name='talib',
    input_names=['close'],          # Dynamique selon func_flags
    param_names=['timeperiod'],    # Dynamique selon introspection
    output_names=['real']          # Dynamique
).from_apply_func(
    talib_func_wrapper,           # Wrapper vers talib.RSI, etc.
    keep_pd=True,
    param_product=True             # Activation du produit cartésien
)
```

2.2. Liaisons Cython et Performance

Bien que `talib-python` soit déjà un wrapper C, l'appel dans une boucle Python est lent.

- **Optimisation :** Utiliser les interfaces "Abstract API" de TA-Lib si disponibles, ou compiler le wrapper `apply_func` via Numba avec `objmode` si nécessaire, bien que Vectorbt gère déjà le broadcasting efficacement via NumPy.

3. GESTION DE L'EXPLOSION COMBINATOIRE

L'optimisation vectorielle génère une matrice de dimension $(T \times P)$, où T est le nombre de timestamps et P le nombre de combinaisons de paramètres. Pour `RSI(period=[2..100])` sur 1 an de données 5m (~100k lignes) : $100k \times 99 \approx 10^7$ points. Gérable. Pour `MACD(fast=[2..50], slow=[2..100], signal=[2..20])` : $100k \times 48 \times 98 \times 18 \approx 8.4 \times 10^9$ points.

Saturation RAM critique.

3.1. Stratégies de Mitigation Mandataires

1. Chunking Paramétrique (Batch Processing) :

- Ne pas calculer toutes les colonnes en une passe.
- Diviser l'espace des paramètres en `chunks`.
- Calculer -> Évaluer Performance -> Garder Top N -> Libérer RAM -> Chunk suivant.

2. Réduction de l'Espace de Recherche (Striding) :

- Ne pas itérer par pas de 1.
- Exemple MACD : `fast=range(2, 50, 5)`.

3. Type Downcasting :

- Forcer les outputs en `float32` au lieu de `float64` (Gain RAM 50%).
- Configuration Vectorbt : `vbt.settings.array_wrapper['dtype'] = np.float32`.

4. Early Pruning (Optuna Integration) :

- Au lieu du `GridSearch` complet (Brute force), utiliser l'échantillonneur TPE (Tree-structured Parzen Estimator) d'Optuna.
- Vectorbt n'exécute que les combinaisons suggérées par l'agent Optuna, réduisant la complexité de $O(N^k)$ à $O(N_{trials})$.